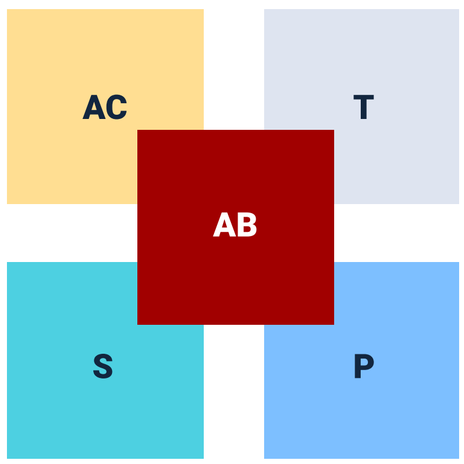
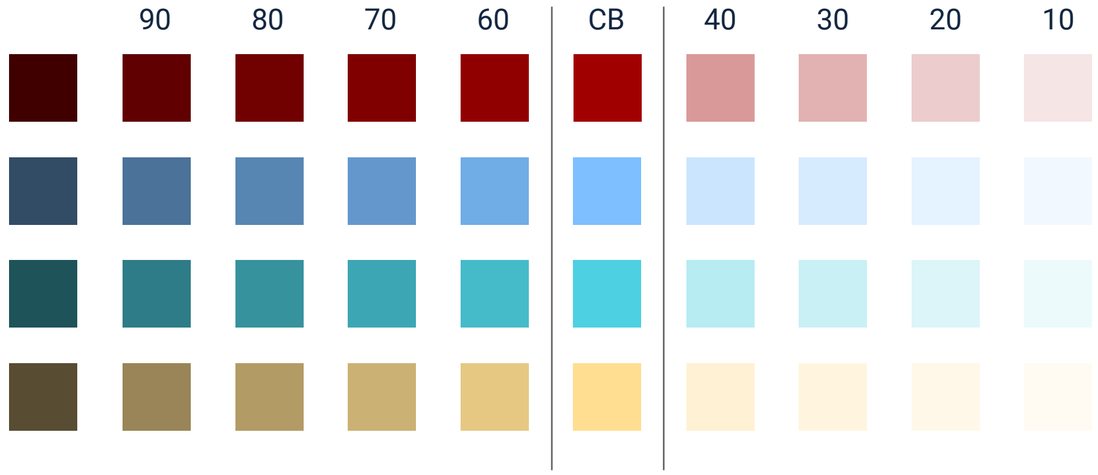
Codificación de algoritmos

Este componente introduce los principios de la programación, los tipos de lenguajes y el uso de entornos de desarrollo. Aborda la sintaxis y estructuras de JavaScript, el manejo de datos, algoritmos básicos de búsqueda y ordenamiento, y el proceso de depuración, identificando errores de sintaxis, lógica y su adecuado tratamiento.

Ver más





1- Primario	#D5AAFF
1- VC	#F1E2FF
1- V	#DDA2FF
1- VO	#62249E
2- Secundario	#9ECFFF
2- VC	#DFEFFF
2- V	#7DBFFF
2- VO	#19538D
3- Acento contenido	#FFDE92
3- VC	#FFEBBE
3- V	#FBB03F
3- VO	#7A4F1E
4- Acento BOTÓN	#A10000
4 - VC	#F9E7E7
5- Terciario	#FFFA80
5- VC	#FFFDCE

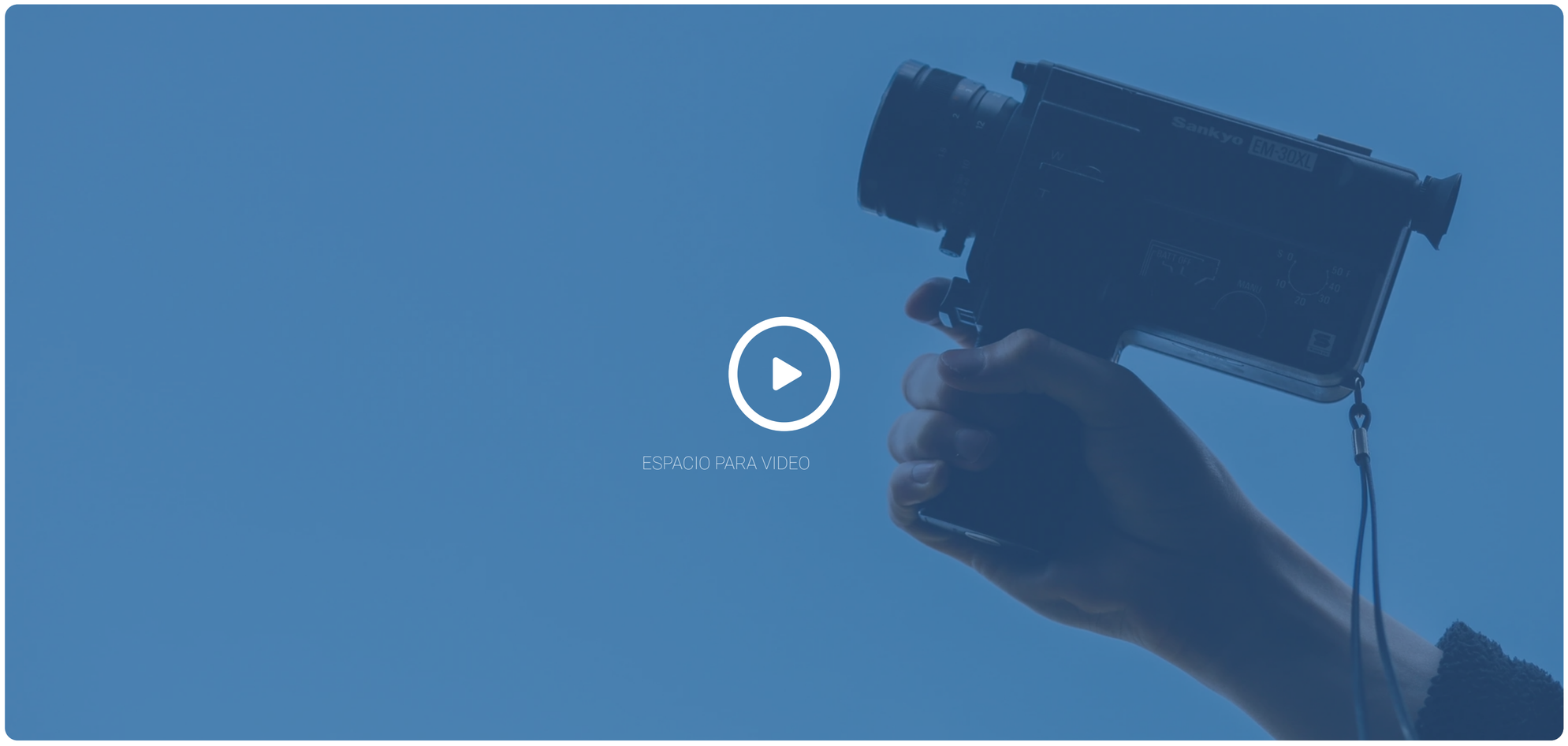
VC = Variable clara
V= Variable
VO= Variable Oscura

Introducción

Los fundamentos de programación abarcan los principios básicos para desarrollar software, incluyendo conceptos como variables, algoritmos y control de flujo, así como la diferencia entre lenguajes compilados (que se traducen antes de ejecutarse) e interpretados (que se ejecutan línea por línea).

Adicionalmente, los entornos de desarrollo permiten escribir y organizar código mediante herramientas como editores o IDEs, junto con la instalación y configuración necesarias y la creación de proyectos; en JavaScript, la sintaxis define el uso de tipos de datos, operadores, expresiones, funciones y comentarios, así como estructuras de selección (if, switch), repetición (for, while) y salto (break, continue, return).

Finalmente, las estructuras de datos y algoritmos permiten organizar y procesar información mediante vectores, matrices, registros, métodos de ordenamiento y técnicas de búsqueda; finalmente, la depuración y manejo de errores incluye la identificación de fallas de sintaxis y lógica, así como el uso de mecanismos como try-catch para controlar excepciones durante la ejecución del programa.



1 Fundamentos de programación

Los fundamentos de programación constituyen la base esencial para el desarrollo de software o sistemas de información, permitiendo a los programadores comprender cómo diseñar, estructurar y ejecutar soluciones computacionales eficientes según la necesidad del cliente. Estos fundamentos incluyen conceptos clave como algoritmos, estructuras de control, tipos de datos y lógica de programación, los cuales son necesarios para resolver problemas de manera sistemática.

Uno de los elementos principales en la programación es el algoritmo, que se define como una serie de pasos ordenados y finitos que permiten resolver un problema específico. Los algoritmos deben ser claros, precisos y eficientes, ya que de ellos depende el rendimiento del programa.

En la programación también se utilizan variables, que son espacios en memoria donde se almacenan datos que pueden cambiar durante la ejecución del programa. Estas variables tienen tipos de datos específicos como números, cadenas de texto o valores booleanos.

Las estructuras de control permiten dirigir el flujo del programa. Entre ellas se encuentran:

Estructuras secuenciales
Conjunto de instrucciones que se ejecutan de manera continua y ordenada, una después de la otra, siguiendo el flujo natural del programa.

Estructuras condicionales
Permiten evaluar condiciones y tomar decisiones, ejecutando diferentes acciones según se cumpla o no una condición (if, else).

Estructuras repetitivas
Facilitan la ejecución de un bloque de instrucciones varias veces, mientras se cumpla una condición o durante un número definido de iteraciones (for, while).

Otro aspecto importante es la sintaxis, que corresponde a las reglas que definen cómo se debe escribir el código en un lenguaje de programación. Una sintaxis incorrecta genera errores que impiden la ejecución del programa.

La lógica de programación es fundamental, ya que permite construir soluciones coherentes. Un programa puede estar bien escrito sintácticamente, pero si su lógica es incorrecta, no producirá los resultados esperados.

Los lenguajes de programación se clasifican en:

Compilados
Necesitan que el código fuente sea traducido completamente a código máquina antes de ejecutarse. Este proceso genera un archivo ejecutable, mejora el rendimiento del programa y permite detectar muchos errores antes de la ejecución.

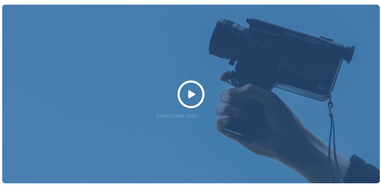
Interpretados
Se ejecutan a través de un intérprete que traduce y ejecuta el código instrucción por instrucción en tiempo real. Esto facilita la prueba, la corrección de errores y la adaptación del programa a distintos entornos.

El desarrollo de habilidades en programación también implica el uso de buenas prácticas como la documentación del código, el uso de nombres descriptivos para variables y funciones, y la organización adecuada del proyecto.

Finalmente, comprender los fundamentos de programación es esencial para avanzar hacia temas más complejos como estructuras de datos, desarrollo web, inteligencia artificial y sistemas distribuidos.

1.1 Conceptos básicos de programación

Para desglosar un poco esta temática, se invita a acceder al siguiente video introductorio, en el cual se presentan de manera general los conceptos básicos de la programación. A través de explicaciones claras y ejemplos sencillos, se aborda qué es un programa, cómo se estructuran los algoritmos y cuál es el papel de las variables y los datos, proporcionando una base conceptual para el estudio de la programación.



Ejemplos de tipos de datos que existen en lenguajes de programación son:

Tabla 1. Tipos de datos

Tipo de dato	Representación	Tamaño (bytes)	Rango de valores	Valor por defecto	Clase asociada
byte	Numérico entero con signo.	1	-128 a 127	0	byte

Los operadores permiten realizar operaciones sobre los datos; existen operadores aritméticos, relacionales y lógicos que permiten construir expresiones más complejas.

Estos son algunos ejemplos:

1. **Operadores aritméticos**
   Se utilizan para realizar operaciones matemáticas básicas.
   Suma (+) — 5 + 3 = 8.
   Resta (-) — 5 - 3 = 2.
   Multiplicación (*) — 5 * 3 = 15.
   División (/) — 6 / 2 = 3.
   Módulo (%) — 5 % 2 = 1.
2. **Operadores relacionales**
3. **Operadores lógicos**

Por su parte, las estructuras de control son fundamentales para dirigir el flujo del programa. Estas incluyen estructuras condicionales como:

1. **If**
   1. INICIO
   2. Leer numero
   3. SI numero > 0 ENTONCES
   4. Mostrar "El numero es positivo"
   5. FIN SI
   6. FIN
2. **For**
3. **While**

Además, la programación requiere seguir buenas prácticas como la correcta indentación del código, el uso de nombres significativos y la documentación mediante comentarios.

Finalmente, comprender estos conceptos básicos es esencial para avanzar hacia temas más complejos en ingeniería de software. Es importante tener en cuenta los elementos de la programación:

El aprendizaje de la programación también implica desarrollar habilidades de pensamiento lógico y resolución de problemas.

Se debe tener presente:

- Practicar constantemente para afianzar los conocimientos adquiridos.
- El uso de diagramas de flujo puede ayudar a visualizar la lógica de un algoritmo antes de implementarlo.
- Los errores son parte del proceso de aprendizaje y permiten mejorar las habilidades del programador.
- El trabajo en equipo y el uso de herramientas colaborativas son fundamentales en proyectos de software.
- Comprender estos conceptos facilita el aprendizaje de cualquier lenguaje de programación.
- La abstracción permite simplificar problemas complejos en partes manejables.
- La programación es una habilidad clave en la transformación digital actual.
- El uso de estándares mejora la calidad del código.
- La revisión de código permite detectar errores y mejorar soluciones.
- El aprendizaje de la programación también implica desarrollar habilidades de pensamiento lógico y resolución de problemas.

1.2 Tipos de lenguajes de programación: compilados e interpretados

En el ámbito de la ingeniería de software, los lenguajes de programación se clasifican comúnmente según la forma en que su código es ejecutado. Dos de las categorías más importantes son los lenguajes compilados e interpretados. Esta clasificación influye directamente en el rendimiento, portabilidad, seguridad y proceso de desarrollo del software.

Lenguajes compilados

Este tipo de lenguaje es aquel en el que el programa escrito por el desarrollador (código fuente) se traduce completamente a lenguaje máquina antes de ejecutarse, mediante una herramienta llamada compilador. Este proceso genera un archivo ejecutable que puede correr directamente en el sistema sin necesidad del código original.

A diferencia de otros tipos de lenguajes, en los compilados la traducción ocurre una sola vez antes de la ejecución, lo que permite que el programa funcione de manera más rápida y eficiente, ya que el computador entiende directamente las instrucciones generadas.

Sus características son:

- El código fuente se traduce completamente a lenguaje máquina antes de ejecutarse.
- Se genera un archivo ejecutable independiente del código original.
- Ofrecen mayor rendimiento y eficiencia en tiempo de ejecución.
- Detectan muchos errores en tiempo de compilación.

Ejemplos: C, C++, Go, Rust.

El proceso de compilación transforma el código escrito por el programador en instrucciones que el procesador puede ejecutar directamente. Esto permite que los programas sean rápidos y eficientes, especialmente en sistemas donde el rendimiento es crítico.

Lenguajes interpretados

Un lenguaje interpretado es aquel en el que el código fuente se ejecuta línea por línea en tiempo real, mediante un programa llamado intérprete, sin necesidad de generar previamente un archivo ejecutable.

En este tipo de lenguajes, el código no se traduce completamente antes de ejecutarse, sino que el intérprete va leyendo, traduciendo y ejecutando cada instrucción al momento. Esto permite una mayor flexibilidad y facilita la prueba y corrección de errores durante el desarrollo.

Por ejemplo, en lenguajes como JavaScript o Python, se puede escribir una instrucción y ejecutarla inmediatamente sin necesidad de compilar todo el programa.

Sus características son:

- El código se ejecuta línea por línea mediante un intérprete.
- No se genera un archivo ejecutable independiente.
- Son más flexibles y fáciles de depurar.
- Permiten mayor portabilidad entre sistemas.

Ejemplos: Python, JavaScript, Ruby.

En los lenguajes interpretados, el intérprete analiza y ejecuta el código en tiempo real. Esto facilita la experimentación y el desarrollo rápido, aunque puede implicar menor rendimiento en comparación con los compilados.

La siguiente tabla relaciona un poco cada proceso que diferencia cada uno de estos lenguajes:

Tabla 2. Comparación de lenguajes de programación

Característica	Compilados	Interpretados	Impacto
Ejecución	Previo a ejecución	En tiempo real	Velocidad vs. flexibilidad

La elección entre un lenguaje compilado o interpretado depende del contexto del proyecto, incluyendo requisitos de rendimiento y velocidad de desarrollo; además, se debe tener en cuenta lo siguiente:

En sistemas embebidos y videojuegos, los lenguajes compilados suelen ser preferidos por su eficiencia.

En aplicaciones web y análisis de datos, los lenguajes interpretados ofrecen mayor productividad.

El uso de entornos de ejecución como máquinas virtuales permite mejorar la portabilidad del software.

El proceso de desarrollo también cambia: los lenguajes interpretados permiten pruebas rápidas sin necesidad de compilación.

Los compiladores modernos también incluyen herramientas de análisis estático para mejorar la calidad del código; comprender estas diferencias permite seleccionar la mejor tecnología para cada tipo de solución.

La elección del lenguaje impacta la arquitectura del sistema, los tiempos de entrega y el mantenimiento del software, por lo que es fundamental analizar las características técnicas antes de decidir.

2 Entornos de desarrollo y configuración del proyecto

En la ingeniería de software, los entornos de trabajo y la correcta configuración de un proyecto son fundamentales para crear aplicaciones robustas, mantenibles y escalables. Estas plataformas no solo facilitan la escritura de código, sino que incorporan herramientas para depurar, probar, administrar dependencias y gestionar versiones. Una configuración adecuada asegura el funcionamiento coherente de los componentes en contextos como prueba, implementación y producción.

Un entorno de desarrollo es el conjunto de herramientas que permiten a los programadores crear software de forma eficiente. Estos entornos pueden variar desde editores simples hasta plataformas altamente integradas, encontrando:

IDE (Entorno de Desarrollo Integrado)

Incluyen herramientas completas como depuradores, compiladores y gestión de proyectos (IntelliJ, Eclipse, Visual Studio).

Editores de código

Son ligeros y extensibles mediante plugins (Visual Studio Code, Sublime Text).

2.1 Entornos de codificación

Constituyen un componente esencial en el desarrollo de software moderno. Proporcionan un conjunto de herramientas que permiten a los desarrolladores escribir, analizar, depurar y mantener código de forma eficiente. La elección adecuada de un entorno impacta directamente en la productividad, la calidad del software y la colaboración en equipos de desarrollo.

Dentro de sus funcionalidades clave se destacan:

- Resaltado de sintaxis que mejora la legibilidad.
- Autocompletado inteligente que reduce errores.
- Depuración paso a paso.
- Integración con control de versiones.

En cuanto al soporte para extensiones, estas funcionalidades permiten optimizar el proceso de desarrollo, reducir errores y mejorar la comprensión del código. A continuación, una breve relación de esta codificación:

Tabla 3. Entornos de codificación

Tipo	Ejemplo	Ventajas	Limitaciones
Editor	VS Code	Ligero, rápido	Configurar extensiones.

Sus buenas prácticas en el uso de entornos se dan al momento de:

- Seleccionar el entorno según el tipo de proyecto.
- Mantener herramientas actualizadas.
- Utilizar control de versiones.
- Evitar instalar extensiones innecesarias.
- Documentar la configuración.

Los entornos de codificación influyen directamente en la productividad del desarrollador. Un entorno bien configurado facilita la detección de errores, mejora la organización del código y permite una integración eficiente con otras herramientas del ciclo de desarrollo.

2.2 Instalación y configuración del entorno

JavaScript es uno de los lenguajes más utilizados en el desarrollo web moderno. Para trabajar de manera eficiente, es fundamental contar con un entorno de codificación correctamente instalado y configurado. Los siguientes son unos pasos necesarios para preparar un entorno profesional de desarrollo en JavaScript:

a) Instalación de herramientas básicas

- Instalar Node.js: esto permite ejecutar JavaScript fuera del navegador.
- Instalar npm (gestor de paquetes incluido con Node.js).
- Instalar un editor de código como Visual Studio Code.
- Instalar Git para control de versiones.

Node.js es esencial porque proporciona el entorno de ejecución para JavaScript en el servidor. Además, npm permite instalar librerías necesarias para el desarrollo.

Visual Studio Code

Como apoyo a este contenido, el siguiente video muestra paso a paso cómo instalar y configurar Java en Visual Studio Code, explicando la preparación del entorno y la integración de las herramientas necesarias para comenzar a programar desde el editor.

Abrir video

b) Configuración del entorno

- Verificar instalación con comandos: node -v y npm -v.
- Configurar variables de entorno si es necesario.
- Instalar extensiones en VS Code (ESLint, Prettier, Live Server).
- Configurar formato automático de código.
- Inicializar proyecto con npm init.

c) Buenas prácticas en JavaScript

- Usar ESLint para detectar errores.
- Formatear código con Prettier.
- Usar control de versiones con Git.
- Organizar el proyecto correctamente.
- Actualizar dependencias regularmente.

Un entorno de JavaScript bien configurado mejora la productividad, permitiendo a los desarrolladores trabajar de manera más eficiente y profesional.

Como base de apoyo a los procesos de instalación de JavaScript se presentan las siguientes guías:

JavaScript Windows

Este documento presenta una guía sencilla para realizar la descarga manual e instalación de Java en Windows, explicando los pasos básicos para obtener el instalador oficial, ejecutarlo correctamente y verificar que la instalación se haya completado con éxito.

Enlace web

JavaScript macOS

Este documento relaciona una guía de ayuda sobre instalación y uso de Java en macOS, donde se explica cómo obtener Java para Mac, los requisitos del sistema, las instrucciones para instalarlo y actualizarlo, así como otros aspectos como activar o desinstalar Java y cómo comprobar la versión instalada, todo desde la documentación oficial de Java.com.

Enlace web

2.3 Creación de proyectos de programación

La creación de proyectos requiere de la identificación de requerimientos o requisitos de un sistema de información del cual se describen los servicios que ha de ofrecer; las restricciones asociadas a su funcionamiento; es decir, las propiedades o restricciones que deben estar registradas, determinadas de forma precisa.

Los siguientes son tres aspectos informativos a tener en cuenta:

Definición general del software

Consiste en describir de forma clara el sistema de información o la solución a desarrollar, explicando la idea central, la funcionalidad principal y el alcance del proyecto. Incluye los propósitos, objetivos generales y el contexto en el que será utilizado, permitiendo comprender su utilidad y valor dentro del entorno para el cual fue concebido.

1/2

A continuación, se presenta una descripción integrada de los elementos clave que conforman la documentación de un proyecto de software, abordando de manera progresiva su definición, desarrollo, implementación y validación:

1. **Definición del proyecto de software**

En primer lugar, en la definición del proyecto de software se debe describir de forma clara la idea general, explicando la funcionalidad principal del sistema de información y el problema que busca resolver. Asimismo, es necesario establecer los objetivos del desarrollo, indicando la necesidad que cubre el sistema, e identificar a los usuarios o entidades que interactuarán con él, junto con el nivel de experiencia del público al que está dirigido el informe.

2. **Especificación de requerimientos**

A partir de esta base, la especificación de requerimientos debe detallar las pautas generales que rigen el funcionamiento del proyecto y los requisitos funcionales que describen los servicios, tareas y comportamientos que el sistema debe realizar. De igual manera, se debe aclarar la autoría del software, indicando si se trata de un desarrollo original o si forma parte de un sistema preexistente, además de definir los alcances y limitaciones del sistema en coherencia con los objetivos establecidos.

3. **Procedimientos de desarrollo**

Posteriormente, en los procedimientos de desarrollo del sistema de información, se describen las herramientas y tecnologías empleadas, tales como entornos de desarrollo integrados, plataformas y lenguajes de programación necesarios para la implementación. Junto con ello, se presenta una planificación general que expone la metodología utilizada y los principales pasos seguidos durante la ejecución del proyecto.

4. **Procedimientos de instalación y prueba**

Finalmente, los procedimientos de instalación y prueba deben contemplar los requisitos no funcionales que afectan el desempeño del sistema, como restricciones relacionadas con tiempos de respuesta. Además, se debe incluir una sencilla para la obtención e instalación del software, dirigida al nivel de usuario definido previamente, así como las especificaciones técnicas de la plataforma y los entornos requeridos para la prueba y ejecución del sistema.

Arquitectura del sistema

Un software de tamaño pequeño está compuesto por varios módulos o partes interconectados. La especificación de la arquitectura del sistema de información define cuáles son estas partes, qué rol tienen dentro del software y cómo se organizan e interconectan.

La información sobre la arquitectura debe incluir como mínimo:

1. **Descripción jerárquica**

Indica de qué forma se organizan jerárquicamente los componentes del sistema de información; es decir, anotar si los mismos están organizados en paquetes, espacios de nombres o bien si el software posee una estructura monolítica.

2. **Diagrama de módulos**

3. **Descripción individual de los módulos**

No es el objetivo de esta sección dar detalles de cómo se realiza la implementación de los módulos, sino únicamente dar una idea generalizada de la existencia de los módulos que hacen parte del sistema.

Diseño del modelo de datos

Es donde se deben identificar cuáles son los agentes involucrados en el sistema de información y nombrarlas en un formato de lenguaje agnóstico. El siguiente es un ejemplo de este lenguaje:

Descripción

El sistema debe calcular el total de una compra multiplicando el precio de un producto por la cantidad seleccionada.

Pseudocódigo

```
INICIO
  LEER precio
  LEER cantidad
  Total= precio * cantidad
  MOSTRAR total
FIN
```

Puede ser un diseño orientado a objetos, relacional, etc., se debe tener una idea general del módulo de datos: entidades, atributos y las relaciones entre ellas. Por consiguiente, es imprescindible incluir diagramas o gráficos que ayuden a visualizar y entender el modelo.

Un programa, aplicación o librería puede a su vez trabajar con varios tipos de datos (entrada, internos o de salida).

Descripción de procesos y servicios ofrecidos por el sistema

Se debe explicar de manera clara cuáles son las tareas o servicios que el sistema ofrece y los procesos que ejecuta para cumplirlos, con el fin de que el lector comprenda su funcionamiento general y la forma en que pueden utilizarse. Para apoyar esta explicación, es conveniente emplear recursos como pseudo-algoritmos o diagramas de flujo que representen el comportamiento del sistema. No se espera mostrar el código ni detallar funciones específicas, sino describir brevemente qué hace cada proceso, su propósito y los datos de entrada y salida, indicando su tipo, cantidad y significado.

En este punto es fundamental que el código fuente de la aplicación esté adecuadamente documentado mediante comentarios claros y pertinentes. Estos constituyen la base de la documentación del proyecto y deben permitir comprender, a un nivel general, el funcionamiento de los procesos y servicios del sistema, así como la estructura y propósito de sus funciones, subrutinas, módulos o clases.

Documentación técnica - Especificación API

Se indica el propósito y breve descripción de cada método/función, con su prototipo indicando argumentos (nombre, tipo, propósito de cada uno) y respuesta (tipo, descripción).

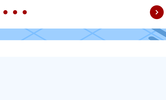
Para llevar a cabo esta tarea, es posible utilizar una variedad de herramientas de generación de documentación automática, a partir del código en el encabezado de cada función (Javadoc, PHPDoc, Doxygen, etc).

La documentación técnica debe pensarse como el manual del programador y ayudar a aquellas personas que estarán a cargo de mantener, ampliar o crear un proyecto derivado a partir de nuestro proyecto.

Para finalizar, se dan los siguientes aspectos relevantes que se deben tener presentes:

Guía de uso del programa

Indicar de forma clara cómo ejecutar el programa, especificando los parámetros obligatorios y opcionales, la función de cada uno y el comportamiento por defecto cuando no se proporcionan. Esta información constituye la base del manual de usuario final de la aplicación.



3 Sintaxis y estructuras de programación en JavaScript

JavaScript es un lenguaje de programación que sostiene los lenguajes HTML & eNaB, estructurales, interactividad y efectos visuales dinámicos. Permite una respuesta inmediata en el navegador, lo que significa que cualquier acción o cambio se refleja de forma automática en la web. Con JavaScript se pueden crear interfaces dinámicas, validar formularios, animar elementos y cargar contenido sin necesidad de recargar la página. Además, es un lenguaje ligero y fácil de aprender para principiantes, pero a la vez suficientemente completo como para desarrollar aplicaciones avanzadas, multiplataforma y escalables.

La siguiente imagen ejemplifica este proceso:

Figura 1. Interpretación Figura

Mi Cuenta
Inicio
Productos
Pagos
Cuenta bancaria
Cerrar sesión
SEGURA
Información importante
Tu cuenta será bloqueada por inactividad en 5 días.
Actualiza tus datos de seguridad.
Acceso a Información
Información personal
Información financiera
Información de seguridad
Información de contacto
Información de inversión
Información de beneficiarios
Información de cuentas
Información de tarjetas
Información de préstamos
Información de seguros
Información de impuestos
Información de jubilación
Información de salud
Información de educación
Información de viajes
Información de entretenimiento
Información de compras
Información de alimentación
Información de transporte
Información de vivienda
Información de mascotas
Información de hobbies
Información de deportes
Información de tecnología
Información de moda
Información de belleza
Información de salud mental
Información de bienestar
Información de espiritualidad
Información de relaciones
Información de familia
Información de amigos
Información de trabajo
Información de estudios
Información de proyectos
Información de metas
Información de logros
Información de fracasos
Información de aprendizaje
Información de crecimiento
Información de cambio
Información de transformación
Información de innovación
Información de creatividad
Información de inspiración
Información de motivación
Información de pasión
Información de propósito
Información de significado
Información de valor
Información de ética
Información de moral
Información de integridad
Información de honestidad
Información de confianza
Información de respeto
Información de empatía
Información de compasión
Información de amor
Información de gratitud
Información de perdón
Información de reconciliación
Información de paz
Información de felicidad
Información de alegría
Información de satisfacción
Información de plenitud
Información de realización
Información de éxito
Información de fracaso

4 Estructuras de datos y algoritmos básicos

Las estructuras de datos y los algoritmos son conceptos fundamentales en programación, ya que permiten gestionar y procesar la información de forma eficiente para resolver problemas. Las estructuras de datos definen cómo se disponen y administran los datos dentro de un programa, facilitando su acceso y uso adecuado.

A continuación, se presentan algunas de las principales estructuras de datos utilizadas en programación, con una breve descripción y ejemplos sencillos que permitirán comprender su funcionamiento y aplicación práctica en la gestión de la información:

Arreglos (arrays)

Permiten almacenar múltiples valores en una sola variable.

Ejemplo:
```
let numeros = [1, 2, 3, 4];
```

Objetos (objects)

Permiten almacenar información en pares clave-valor.

Ejemplo:
```
let persona = {
  nombre: 'Juan',
  edad: 25
};
```

Pilas (stacks)

Siguen el principio LIFO (último en entrar, primero en salir).

Ejemplo:
```
let pila = [];
pila.push(1);
pila.pop();
```

Colas (queues)

Siguen el principio FIFO (primero en entrar, primero en salir).

Ejemplo:
```
let cola = [];
cola.push(1);
cola.shift();
```

De igual manera, se presentan algunos algoritmos básicos ampliamente utilizados en programación, orientados a la búsqueda, organización y recorrido de datos, con ejemplos que facilitan la comprensión de su lógica y aplicación práctica:

Búsqueda

Permite encontrar un elemento dentro de una lista.

Ejemplo:
```
function buscar(lista, valor) {
  for (let i = 0; i < lista.length; i++) {
    if (lista[i] === valor) return i;
  }
  return -1;
}
```

Ordenamiento

Permite organizar los datos.

Ejemplo:
```
function ordenar(lista) {
  for (let i = 0; i < lista.length; i++) {
    for (let j = 0; j < lista.length - 1 - i; j++) {
      if (lista[j] > lista[j + 1]) {
        let temp = lista[j];
        lista[j] = lista[j + 1];
        lista[j + 1] = temp;
      }
    }
  }
  return lista;
}
```

Recorrido

Consiste en visitar cada elemento de una estructura.

Ejemplo:
```
let numeros = [1,2,3];
numeros.forEach(n => console.log(n));
```

Las estructuras de datos y algoritmos permiten organizar mejor la información, optimizar el rendimiento de los programas y resolver problemas de manera eficiente; comprender las estructuras de datos y los algoritmos es esencial para desarrollar software eficiente y de calidad en JavaScript.

4.1 Estructuras de datos

Generalmente, al desarrollar programas, se declaran e inicializan variables para realizar diferentes operaciones. Estas resultan muy útiles para almacenar de manera temporal valores que serán utilizados durante la ejecución del programa. En esta sección se estudiarán los arreglos, los cuales ofrecen la posibilidad de contar con un contenedor de valores u objetos de un mismo tipo. Son especialmente útiles cuando se requiere almacenar un conjunto de datos homogéneos, como por ejemplo calificaciones, nombres de personas o temperaturas. Sin embargo, pueden ser muy diversos. Sin duda, en muchos casos, el uso de arreglos facilitará las operaciones en los programas que se desarrollen.

Ejemplo:
String[] arreglo1Textos = new String[5];

Para definir un arreglo, se indica el tipo de dato que contendrá. Luego, se agregan corchetes de apertura y cierre [].

En el anterior ejemplo, se utiliza String[] como tipo de arreglo. Posteriormente, se escribe el nombre con el cual se identificará el arreglo. Se añade el operador de asignación =, seguido de la palabra reservada new, y nuevamente el tipo de dato con sus corchetes, pero esta vez indicando dentro de ellos la longitud del arreglo.
Para este caso, new String[7].

Un aspecto importante a considerar es que los arreglos pueden ser de cualquier tipo, incluyendo datos primitivos.

Ejemplo de Arreglos en java

```
public class EjemploArreglo {
    public static void main(String[] args) {

        // Declaración e inicialización de un arreglo
        int[] numeros = {10, 20, 30, 40, 50};

        // Acceso a un elemento del arreglo
        System.out.println("Primer elemento: " + numeros[0]);

        // Modificar un elemento
        numeros[2] = 35;

        // Recorrer el arreglo con un ciclo for
        System.out.println("Elementos del arreglo:");
        for (int i = 0; i < numeros.length; i++) {
            System.out.println(numeros[i]);
        }
    }
}
```

Explicación:

- int[] numeros → Declara un arreglo de enteros.
- {10, 20, 30, 40, 50} → Inicializa el arreglo con valores.
- numeros[0] → Accede al primer elemento (los índices empiezan en 0).
- numeros.length → Obtiene el tamaño del arreglo.
- El for permite recorrer todos los elementos.

Matrices

En JavaScript, una matriz es una estructura de datos bidimensional que puede representarse mediante un arreglo cuyos elementos son otros arreglos. Esta estructura permite organizar la información en filas y columnas, de manera similar a una tabla.

Ejemplo:
```
let matriz = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];
```

Partiendo de lo anterior, se describen los aspectos fundamentales para trabajar con matrices en JavaScript, desde el acceso y recorrido de sus elementos hasta sus aplicaciones más comunes y algunas buenas prácticas que facilitan un uso correcto y eficiente de esta estructura de datos:

- **Acceso a elementos**

Para acceder a un elemento se utilizan dos índices: fila y columna.

Ejemplo:
```
matriz[0][1]; // Accede al valor 2
```

4.2 Métodos de ordenamiento

Los métodos de ordenamiento son algoritmos fundamentales en programación que permiten organizar los elementos de un arreglo en un orden específico, ya sea ascendente o descendente. En el desarrollo de software, el ordenamiento es clave para optimizar búsquedas, mejorar el rendimiento y facilitar el análisis de datos.

Los algoritmos de ordenamiento se utilizan en múltiples escenarios reales, como, por ejemplo:

Listas de usuarios.

Sistemas de reportes.

Procesamiento de datos.

Bases de datos.

Aplicaciones web y móviles.

Un buen algoritmo de ordenamiento puede reducir significativamente el tiempo de ejecución de un programa.

Tipos de algoritmos de ordenamiento

Existen dos grandes categorías:

Algoritmos simples (O(n²))

Fáciles de entender, pero menos eficientes.

Algoritmos eficientes (O(n log n))

Usados en aplicaciones reales.

Bubble Sort (Ordenamiento burbuja)

Este algoritmo compara elementos adyacentes e intercambia sus posiciones si están en el orden incorrecto.

Complejidad: O(n²).

Ejemplo:

```
function burbuja(arr){
  for(let i=0;i<arr.length;i++){
   for(let j=0;j<arr.length-1;j++){
     if(arr[j]>arr[j+1]){
      let temporal=[];
      arr[j]=arr[j+1];
      arr[j+1]=temp;
     }
   }
  }
  return arr;
}
```

Estos algoritmos se dividen de la siguiente manera y se explican sus respectivas acciones:

Selection Sort

Busca el valor mínimo del arreglo y lo coloca en la posición correcta.

Complejidad: O(n²).
Uso: útil para aprendizaje y estructuras pequeñas.

1/2

Basado en lo anterior, se resume y relaciona la comparación de algoritmos:

Burbuja	Selección	Inserción	Quicksort	MergeSort
O(n²) - simple.	O(n²) - básico.	O(n²) - eficiente en datos pequeños.	O(n log n) - rápido en la práctica.	O(n log n) - estable y eficiente.

Método sort() en JavaScript

JavaScript incluye un método nativo para ordenar arreglos:

```
array.sort((a,b)=>a-b);
```

Este método utiliza algoritmos optimizados internamente, por lo que es el más recomendado en la mayoría de los casos.

4.3 Métodos de búsqueda para datos numéricos, textos y registros

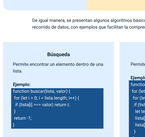
En el desarrollo de software, uno de los problemas más frecuentes consiste en localizar información dentro de un conjunto de datos. Este proceso se conoce como búsqueda y es fundamental en prácticamente cualquier aplicación informática, desde sistemas simples hasta plataformas complejas que manejan grandes volúmenes de información.

En JavaScript, los métodos de búsqueda permiten encontrar elementos específicos dentro de estructuras de datos, como arreglos u objetos. Estos elementos pueden ser de diferentes tipos, tales como datos numéricos, cadenas de texto o registros más complejos (por ejemplo, objetos que representan usuarios, productos o transacciones).

La importancia de los métodos de búsqueda radica en que permiten acceder de manera eficiente a la información, lo cual impacta directamente en el rendimiento y la experiencia del usuario. Por ejemplo, en una aplicación web, cada vez que un usuario busca un producto, filtra resultados o accede a un registro específico, se está utilizando algún tipo de algoritmo o método de búsqueda.

Existen diferentes enfoques para realizar búsquedas, los cuales varían según la estructura de los datos y el contexto del problema. Entre los más comunes se encuentran:

1. Búsqueda lineal, que recorre todos los elementos hasta encontrar el valor deseado.
2. Búsqueda binaria, que es más eficiente, pero requiere que los datos estén previamente ordenados.
3. Métodos nativos de JavaScript como find(), filter(), includes() e indexOf(), que facilitan la búsqueda sin necesidad de implementar algoritmos desde cero.



Cuando se trabaja con datos numéricos, la búsqueda suele centrarse en encontrar valores específicos o rangos dentro de listas. En el caso de datos de texto, es común buscar coincidencias exactas o parciales dentro de cadenas. Por otro lado, cuando se manejan registros (objetos), la búsqueda puede implicar condiciones más complejas, como encontrar un objeto que cumpla ciertos criterios en una o varias de sus propiedades.

Además, en el contexto profesional, no solo es importante encontrar los datos, sino hacerlo de manera eficiente y escalable. A medida que el volumen de información crece, elegir el método de búsqueda adecuado se vuelve crucial para evitar problemas de rendimiento.

En JavaScript moderno, el uso de funciones de orden superior y métodos de los arreglos ha simplificado considerablemente la implementación de algoritmos de búsqueda, permitiendo escribir código más limpio, legible y mantenible. No obstante, comprender los principios que los sustentan sigue siendo fundamental para tomar decisiones informadas y optimizar soluciones en escenarios de mayor complejidad.

En conclusión, los métodos de búsqueda en JavaScript constituyen una herramienta fundamental para el manejo de datos, ya que permiten localizar, filtrar y procesar información de manera eficiente, siendo una base clave en el desarrollo de aplicaciones modernas.

5 Depuración y manejo de errores en programas

En el desarrollo de aplicaciones, los errores y las excepciones son inevitables. Al tener el rol de programadores, se posee la responsabilidad maniqueíos adecuadamente para evitar que la experiencia del usuario se vea afectada. Una correcta gestión de errores también permite a los desarrolladores depurar el código y comprender mejor sus causas para poder solucionarlas de manera eficiente.

JavaScript fue sido un lenguaje de programación ampliamente utilizado durante más de 25 años, tanto en el desarrollo de aplicaciones del lado del cliente (front-end) como del servidor, utilizando diversas bibliotecas populares (como React.JS) y frameworks (como Next.js, Remix, entre otros).

Al ser un lenguaje de tipado dinámico, JavaScript presenta el desafío de manejar correctamente la seguridad de tipos. TypeScript ayuda a controlar este aspecto, pero aun así es necesario gestionar los errores en tiempo de ejecución de manera eficiente desde el código.

Errores como TypeError, RangeError y ReferenceError probablemente sean familiares si se tiene experiencia desarrollando en JavaScript. Estos errores ocurren cuando el código intenta realizar operaciones inválidas, como acceder a propiedades indefinidas o incluso provocar que la aplicación falle completamente, lo cual afecta negativamente la experiencia del usuario final.

5.1 Depuración de código

La *depuración de código* (debugging) es el proceso mediante el cual los desarrolladores identifican, analizan y corrigen errores dentro de un programa. En JavaScript, esta práctica es fundamental debido a que muchos errores se detectan en tiempo de ejecución.

¿Qué es la depuración?

La depuración consiste en detectar errores, analizar el comportamiento del programa y corregir fallas en la lógica del código.

Las siguientes son las herramientas de depuración:

Igualmente, para detectar errores en Java existen unas técnicas de depuración, como son:

5.2 Fallas de síntaxis

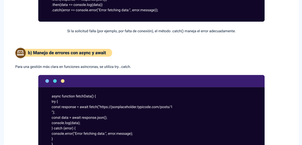
En el desarrollo de software, uno de los errores más comunes que enfrentan los programadores son las fallas de síntaxis, también conocidas como SyntaxError. Estas ocurren cuando el código escrito no cumple con las reglas gramaticales del lenguaje de programación, impidiendo que el intérprete o el motor de JavaScript pueda comprenderlo y ejecutarlo correctamente el programa.

JavaScript, al ser un lenguaje interpretado, analiza el código antes de ejecutarlo. Durante este proceso, si detecta una estructura incorrecta — como paréntesis sin cerrar, comillas mal utilizadas, palabras reservadas mal escritas o una mala organización del código — genera un error de síntaxis y detiene la ejecución del programa. Esto significa que, a diferencia de otros errores, las fallas de síntaxis deben corregirse obligatoriamente antes de que el código pueda ejecutarse.

Las fallas de síntaxis pueden presentarse en diferentes formas, tales como:

- Falta de punto y coma al finalizar instrucciones.
- Uso incorrecto de llaves {} o paréntesis ().
- Declaraciones incompletas o mal estructuradas.
- Errores en cadenas de texto (comillas abiertas o mal cerradas).
- Uso incorrecto de palabras reservadas del lenguaje.

Para la síntaxis de fallas, se ejemplifica con los siguientes código de programación:



A continuación, se presentan distintas formas y escenarios prácticos para generar, gestionar y clasificar errores, desde la creación de errores personalizados hasta el uso de estructuras modernas para el control de fallas y la identificación de los tipos de errores más comunes en JavaScript.

a) Generar errores personalizados

Es posible generar errores de forma manual utilizando la instrucción throw.

```
function checkAge(age) {
  if (age < 18) {
    throw new Error("No mayor de 18 de edad.");
  }
  return "Acceso granted";
}

try {
  console.log(checkAge(16));
} catch (error) {
  console.log("Error: " + error.message);
}
```

¿Cuando usar el manejo de errores de JavaScript?

El manejo de errores en JavaScript es fundamental en situaciones donde pueden presentarse problemas inesperados. Algunos casos comunes son:

Se relacionan algunos ejemplos de manejo de errores en JavaScript, partiendo de lo siguiente:

¿Cómo manejar errores en código asincrono? Cuando se trabaja con funciones asincronas y promesas, los errores pueden gestionarse usando .catch().

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error("Error fetching data", error.message));
```

Si la solicitud falla (por ejemplo, por falta de conexión), el método .catch() maneja el error adecuadamente.

b) Manejo de errores con async y await

Para una gestión más clara en funciones asincronas, se utiliza try...catch:

```
async function fetchData() {
  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/posts/1");
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error("Error fetching data", error.message);
  }
}

fetchData();
```

Esta extra promesas no controladas y permite un manejo estructurado de errores.

c) Utilizar finally para limpieza

El bloque finally asegura que ciertas acciones se ejecuten siempre, incluso si ocurre un error.

```
function processTask() {
  try {
    console.log("Processing...");
    throw new Error("Something went wrong");
  } catch (error) {
    console.error("Caught error", error.message);
  } finally {
    console.log("Task completed (cleaning up resources).");
  }
}

processTask();
```

d) Diferentes tipos de errores de JavaScript

Los errores en JavaScript se clasifican en distintas categorías:

5.3 Fallas de lógica

Las fallas de lógica en JavaScript son errores que no impiden que el programa se ejecute, pero sí producen resultados incorrectos, inconsistentes o distintos a los esperados. A diferencia de los errores de síntaxis, estas fallas no siempre muestran un mensaje en pantalla, por lo que suelen ser más difíciles de detectar. En la práctica profesional, muchos de los errores que parecen ser "la lógica del programa, las condiciones, los cálculos o la manipulación de datos no reflejan correctamente la intención del desarrollador.

A continuación, se presentan algunas de las fallas lógicas más comunes en JavaScript, organizadas en distintos escenarios que suelen afectar el flujo del programa, la correcta evaluación de condiciones y el resultado final de las operaciones. El objetivo de este material, accesiblepara desarrolladores principiantes, con el fin de ayudarlos a identificar, comprender y prevenir estos problemas durante el desarrollo de aplicaciones.

a) Uso incorrecto de operadores

Una de las fallas más comunes en JavaScript ocurre al utilizar operadores equivocados dentro de una condición o una expresión. Si usas más frecuente es usar el operador de asignación (=) en lugar de un operador de comparación (== o ===). Cuando esto ocurre, el valor de la variable cambia y la condición puede evaluarse como verdadera, aunque esa no fuera la intención del programador. Este error afecta el flujo lógico del programa y puede provocar comportamientos difíciles de identificar.

Ejemplo:

```
let x = 5;

if (x = 10) {
  console.log("Verdadero");
}

// Error: se usó '=' en lugar de '==' o '==='
// En este caso, la variable x vale 10 y la condición resulta verdadera, por lo que el bloque se ejecuta. La forma correcta sería utilizar un operador de comparación estricta.

// Corrección:
let x = 5;
if (x === 10) {
  console.log("Verdadero");
} else {
  console.log("Falso");
}
```

b) Condiciones mal planteadas

c) Bucles infinitos

d) Comparación de tipos incorrecta

e) Variables no inicializadas

f) Indices fuera de rango

g) Mutación no deseada de datos

h) Retornos incorrectos en funciones

i) Uso incorrecto de condiciones compuestas

j) Confusión entre incremento previo y posterior

5.4 Manejo de errores y excepciones

Para entender estos errores, se presentan dos situaciones comunes en las que el manejo de errores resulta esencial para evitar fallas en la ejecución del programa: el control de operaciones matemáticas inválidas y la gestión de datos incorrectos provenientes de fuentes externas. Estos ejemplos muestran cómo el uso de try...catch permite identificar errores y mantener la estabilidad de la aplicación.

a) Manejo de la división por cero

A continuación, se tiene una función que divide un número entre otro. Para ello, se pasan parámetros a la función para ambos números. Deseando asegurarnos de la división nunca genere un error al dividir un número entre cero (0).

Como medida preventiva, se ha escrito una condición que, si el divisor es cero, generará un error indicando que la división por cero no está permitida. En cualquier otro caso, se procederá con la división. Si se produce un error, el bloque catch lo gestionará y mostrará los avisos oportunos (en este caso, registrar el error en la consola).

```
function dividirNumeros(a, b) {
  try {
    if (b === 0) {
      const err = new Error("Division by zero is not allowed.");
      throw err;
    }
    const result = a/b;
    console.log("The result is: ${result}");
  } catch(error) {
    console.error("Can't divide: ", error.message)
  }
}
```

Ahora bien, a se invoca la función con los siguientes argumentos, se obtendrá un resultado de 5, y si segundo argumento es un valor distinto de cero:

```
dividirNumeros(10, 0); // The result is 5
dividirNumeros(10, "cero"); // segundo argumento, el programa generará un error y este se registrará en la consola.

dividirNumeros(10, 5);
```


b) Error matemático

A menudo, se recibirá JSON como respuesta a una llamada a la API. Se necesita analizar este JSON en el código: *JavaScript para extraer los valores.* ¿Qué ocurre si la API no envía un JSON real formado por error? No puede permitirse que la interfaz de usuario falle por esto. Se debe gestionar correctamente y aquí es donde el bloque try...catch suelve a ser la solución.

```
function parseJSONSafely(str) {
  try {
    return JSON.parse(str);
  } catch (err) {
    console.error("Invalid JSON:", err.message);
    return null;
  }
}

const userData = parseJSONSafely('{"name": "JaysSoSpicy"}'); // Parsed
const badData = parseJSONSafely('name: JaysSoSpicy'); // Manifest gracefully
En try...catch, la respuesta formada provocará que la aplicación falle.
```

Tablet

Codificación de algoritmos

Este componente introduce los principios de la programación, los tipos de lenguajes y el uso de entornos de desarrollo. Aborda la sintaxis y estructuras de JavaScript, el manejo de datos, algoritmos básicos de búsqueda y ordenamiento, y el proceso de depuración, identificando errores de sintaxis, lógica y su adecuado tratamiento.

Iniciar >

Movil

Codificación de algoritmos

Este componente introduce los principios de la programación, los tipos de lenguajes y el uso de entornos de desarrollo. Aborda la sintaxis y estructuras de JavaScript, el manejo de datos, algoritmos básicos de búsqueda y ordenamiento, y el proceso de depuración, identificando errores de sintaxis, lógica y su adecuado tratamiento.

Iniciar >

Técnico en programación de aplicaciones y servicios para la nube

Síntesis: Codificación de algoritmos

El componente formativo aborda de manera integral los fundamentos de la programación, iniciando con los conceptos básicos y la comprensión de los lenguajes compilados e interpretados. Posteriormente, se estudian los entornos de desarrollo, la instalación y configuración del ambiente de trabajo y la creación de proyectos, estableciendo una base sólida para la codificación. A continuación, se profundiza en la sintaxis y las estructuras de programación en JavaScript, incluyendo tipos de datos, operadores, funciones, estructuras de control, repetición y salto, que permiten construir soluciones lógicas y estructuradas. Asimismo, se introducen las estructuras de datos y los algoritmos básicos de ordenamiento y búsqueda, aplicados a diferentes tipos de información. Finalmente, el componente desarrolla habilidades para la depuración de código, la identificación de fallas de sintaxis y lógica, y el manejo de errores y excepciones, fortaleciendo la capacidad de crear programas confiables, eficientes y mantenibles.